What is a Genetic Algorithm?

A **Genetic Algorithm** is an optimization and search technique inspired by **natural evolution**.

In nature, stronger organisms are more likely to survive and reproduce. Over many generations, useful traits become more common in the population.

A genetic algorithm applies the same idea computationally:

> Start with many possible solutions, evaluate them, keep the better ones, combine them, randomly modify them, and repeat until a good solution emerges.

GAs are especially useful when the search space is large, complex, non-linear, or difficult to solve using exact mathematical methods.

Basic Intuition

The Problem

Suppose we want to solve an optimization problem.

The Population

Starts with a **population** of solutions, instead of just one.

The Individual

Each solution is called an **individual** or **chromosome**.

Fitness Function

Each individual is evaluated using a **fitness function**.

Survival of the Fittest

Better fitness increases the chance to contribute to the next generation.

Improvement

Over time, the population ideally improves.

Representing Solutions

A Genetic Algorithm first needs a way to represent each possible solution. This representation is called a **chromosome**.

Feature Selection

`10110010`

Binary string representation where:
1 = feature selected
0 = feature not selected

Route Optimization

Dhaka → Rajshahi → Khulna → Sylhet

Permutation-based representation of cities to visit in sequence.

Hyperparameter Tuning

`[learning rate, batch size, dropout]`

Value-based representation for optimizing model parameters.

Quality Mandate: A good representation should express valid solutions and allow crossover and mutation to work meaningfully.

Measuring Fitness

The **fitness function** measures how good a solution is and guides the entire evolutionary process.

Route Optimization

`Fitness = 1 / distance`

The goal is to minimize total travel distance. Shorter paths result in higher fitness values.

Machine Learning

`Fitness = Accuracy`

Evaluation is based on validation accuracy. Models that generalize better on unseen data are preferred.

Feature Selection

`Fitness = Acc - Penalty`

Balances model accuracy against the number of selected features to prevent complexity.

A good fitness function should reflect the real goal, distinguish solutions accurately, and avoid misleading the search.

Selecting Parents

After measuring fitness, the GA selects individuals as parents.

Better solutions should have a higher chance of reproduction, but weaker solutions should not be completely ignored.

Common methods:

- Roulette Wheel Selection
  - individuals are selected according to fitness probability. Higher-fitness individuals occupy larger portions of the selection wheel
- Tournament Selection
  - a small group is randomly chosen, and the best among them becomes a parent.
- Rank-Based Selection
  - individuals are ranked, and selection is based on rank rather than raw fitness.

Selection controls how strongly the algorithm favors better solutions.

Crossover and Mutation

Crossover

Combines two parents to produce new offspring.

Example:

```
Parent 1: 101|110
Parent 2: 011|001
```

Result:

```
Child 1: 101001
Child 2: 011110
```

Exploitation:
Helps exploit good existing solutions.

Mutation

Randomly changes part of a solution.

Example:

```
Before: 101100
After : 101000
```

Exploration:
Helps explore new areas and prevents the algorithm from getting stuck.

Balance is Key: Crossover refines solutions while Mutation maintains diversity.

Preserving Diversity

A Genetic Algorithm must balance improvement and diversity. Lack of diversity leads to **premature convergence**.

The Challenge

If individuals become too similar, the GA stops exploring and settles for a poor, local solution.

Signs of Stagnation:

- Population becomes too uniform
- No improvement over many generations

Ways to Preserve Diversity

- **Mutation:** Introduce new genetic material
- **Controlled Elitism:** Keep only a small % of the best
- **Selection Pressure:** Avoid selecting only the best every time
- **Population Size:** Maintain reasonable scale
- **Immigration:** Introduce random individuals when stagnant

Key Message: A good GA improves quality while keeping enough diversity to continue exploring better possibilities.

Swarm Intelligence

Swarm Intelligence is a branch of AI inspired by the collective behavior of simple agents in nature.

Examples in Nature

- Ants finding shortest paths
- Birds flocking
- Fish schooling
- Bees searching for food

The Key Idea

Simple individuals following simple rules can produce **intelligent group-level behavior**.

Emergent Intelligence:

Intelligence emerges from interaction among many agents, with no single central controller.

Decentralized Control: Collective behavior produces complex outcomes from simple interactions.

Why Swarm Intelligence Matters in AI

Many AI problems involve searching for good solutions in a large space.

Examples

- Routing & Scheduling
- Path Planning
- Feature Selection
- Clustering
- Hyperparameter Tuning
- Resource Allocation

Key Benefits

- Population-based
- Flexible & Simple
- Complex Optimization
- Explores Multiple Solutions
- Robust and Efficient

Popular Algorithms

Ant Colony Optimization

Particle Swarm Optimization

Swarm Intelligence: A powerful approach for solving search-intensive AI challenges.

Ant Colony Optimization (ACO)

ACO is inspired by how ants find shortest paths between their nest and food sources.

The Natural Process

- Ants leave behind a chemical substance called **pheromone**.
- Others follow paths with stronger pheromones.
- Shorter/useful paths attract more ants, strengthening the signal.
- The colony collectively discovers the optimal path over time.

Application in AI

Artificial ants build solutions step by step, mimicking biological behavior.

Feedback Loop:

1. **Evaluation:** Good solutions receive stronger artificial pheromone.
2. **Selection:** Future ants are more likely to follow high-pheromone choices.

Collective Discovery: AI agents collaborate to solve complex optimization problems.

Basic ACO Idea

The ACO process can be summarized as a iterative search for optimal solutions.

Summary of the ACO Process

1. Initialize pheromone values.
2. Let many artificial ants construct solutions.
3. Evaluate the quality of each solution.
4. Add more pheromone to good solutions.
5. Evaporate some old pheromone.
6. Repeat until a good solution is found.

Two Important Concepts

Pheromone

Represents learned experience. Stronger pheromones indicate choices that were useful in the past.

Evaporation

Gradually reduces values to prevent dependency on early solutions and maintain exploration.

Balancing Exploration and Exploitation: The core of effective Ant Colony Optimization.

Example: Shortest Path Problem

Suppose ants need to find the shortest route between two cities.

1. Initial Exploration

At first, ants explore different routes randomly without any prior knowledge.

2. Speed of Return

If one route is shorter, ants completing that route return faster and reinforce it more frequently.

3. Pheromone Strength

As a result, that shorter route receives a significantly stronger pheromone trail.

4. Convergence

Eventually, more ants follow that route, and the colony converges toward a good solution.

Optimization through Collective Behavior: Finding the global minimum via local reinforcement.

Particle Swarm Optimization (PSO)

PSO is inspired by the movement of birds or fish searching for food, where each candidate solution is a **particle**.

Movement & Search Space

Each particle moves through the search space, updating its trajectory based on collective and individual intelligence.

- Simulates swarm behavior
- Dynamic position updates
- Iterative optimization

Dual Experience Learning

1. Personal Best

Learning from its **own best solution** found so far.

2. Global Best

Learning from the **best solution** found by the whole swarm.

Collaborative Learning: The particle learns from itself and from the group to reach the goal.

Particle Swarm Optimization (PSO)

How a particle moves

Each particle has two main things:

Position

The current solution of the particle.

Velocity

The direction and speed of movement.

At every iteration, PSO updates the velocity first:

$$v = wv + c_1 r_1 (pbest - x) + c_2 r_2 (gbest -$$

$$x = x + v$$

x	current position/solution
v	velocity/movement direction
$pbest$	particle's own best solution
$gbest$	best solution found by the whole swarm
w	inertia; tendency to continue previous movement
c_1	personal learning factor
c_2	social/global learning factor
r_1, r_2	random values to keep exploration dynamic

Basic PSO Idea

The PSO process is an iterative search inspired by social behavior.

Core Concepts

Position
The current solution (e.g., [learning rate, batch size]).

Velocity
The direction and speed of movement in search space.

Personal & Global Best
Best solutions found by individual and entire swarm.

The PSO Algorithm

- Initialize particles with random position and velocity.
- Evaluate fitness of each particle.
- Update personal best and global best.
- Adjust velocity and move toward promising regions.
- Repeat until stopping criteria are met.

Optimization Strategy: Particles evolve based on individual success and collective intelligence.